Improving Science via Artificial Intelligence
Part 1: Introduction to GenAI and LLMs

Nathaniel E. Helwig

Associate Professor of Psychology and Statistics
Core Member of the Data Science Initiative
University of Minnesota

Quantitative & Psychometric Methods Seminar
University of Minnesota, Twin Cities
Dec 3, 2025

Table of Contents

Table of Contents

Motivation for Talking about GenAI and LLMs

ChatGPT debuted over three years ago (on November 30, 2022), and introduced the world to generative artificial intelligence (GenAI).

Large language models (LLMs) like ChatGPT have immense potential for improving science—and they keep getting better every day!

I went from a skeptic to a believer after attending the Data and AI Intensive Research with Rigor and Reproducibility (DAIR3) program.

Figure 1: Logos for some popular large language models.

Primary and Secondary Goals

The primary goal of this talk series is to encourage you to explore responsible uses of GenAI and LLMs in your research (and teaching!).

Secondary goals include...

- Introducing different GenAI tools/models
- Normalizing the use of GenAI in workflows
- Distinguishing GenAI from its predecessors
- Understanding how LLMs generate text
- Comparing and validating LLM outputs
- Evaluating applications of GenAI in science

In other words, I hope to INDUCE change in how you engage with research, teaching, and other scholarly activities.

Table of Contents

What is Generative Artificial Intelligence (GenAI)?

ChatGPT:

Generative Artificial Intelligence (Generative AI) refers to a class of artificial intelligence models designed to create new content. Unlike traditional AI systems that classify, predict, or make decisions based on existing data, generative AI learns patterns from large datasets and uses that knowledge to produce original outputs.

Claude:

Generative Artificial Intelligence (GenAI) refers to AI systems that can create new content rather than just analyzing or classifying existing data. These systems learn patterns from training data and use that knowledge to generate original outputs like text, images, music, code, or video.

Gemini:

Generative Artificial Intelligence (Generative AI or GenAI) is a type of Artificial Intelligence that can create new and original content in response to a user's prompt or request, which is often provided in natural language.

How Does GenAI Work? (ChatGPT)

Generative AI typically uses advanced machine learning models, such as

- **Large Language Models (LLMs)** like GPT:
  Learn statistical patterns in text and generate new sentences or paragraphs based on prompts.
- **Diffusion Models** (used in modern image generation):
  Start from random noise and iteratively "denoise" it into a coherent image.
- **Generative Adversarial Networks (GANs)**:
  Use two competing neural networks—one generates data, one critiques it—to produce realistic outputs.
- **Variational Autoencoders (VAEs)**:
  Learn latent representations and generate new examples from them.

How Does GenAI Work? (Claude)

Most modern generative AI uses deep learning models, particularly neural networks. For example, large language models like GPT or Claude are trained on vast amounts of text and learn statistical patterns about how language works. Image generators like DALL-E or Midjourney learn from millions of images to understand visual concepts and styles.

How Does GenAI Work? (Gemini)

Generative AI is built upon sophisticated machine learning models, often a type of deep learning model called a neural network. The model is trained on an enormous and diverse dataset (e.g., billions of text documents or images) where it learns the complex patterns and relationships within the data.

Key model types include:

- **Large Language Models (LLMs)**: A class of Foundation Models trained primarily on text data, excelling at language-based tasks like conversation and summarization (e.g., the "GPT" in ChatGPT). These models predict the most probable next word or sequence of words to create coherent text.
- **Generative Adversarial Networks (GANs)**: These models use two neural networks—a Generator that creates new data and a Discriminator that evaluates the data for authenticity. They work against each other to continuously improve the realism of the generated content.
- **Diffusion Models**: These models create new data by starting with pure noise and then iteratively "denoising" it based on the learned data patterns until a realistic image or piece of content is produced.

Table of Contents

Generative versus Discriminative AI (ChatGPT)

Generative AI and discriminative AI are two fundamentally different approaches to modeling data in machine learning. The distinction is about what the model learns and what tasks it is designed to perform.

- Core Difference:
  Discriminative models learn the conditional probability distribution $P(y|x)$
  Generative models learn the joint probability distribution $P(x, y)$
- What They Are Designed to Do:
  Discriminative AI predicts values or labels using $P(y|x)$
  Generative AI produces new data by simulating from $P(x, y)$

Here are two examples to solidify the distinction:

- Email Spam Detection
  Discriminative AI learns $P(\text{spam}|\text{text})$ to classify new emails
  Generative AI learns $P(\text{text}, \text{spam})$ to generate new spam emails
- Images of Cats and Dogs
  Discriminative AI learns $P(\text{animal} \mid \text{image})$ to classify new images of cats/dogs
  Generative AI learns $P(\text{image}, \text{animal})$ to generate new images of cats/dogs

Generative versus Discriminative AI Table (ChatGPT)

Table 1: ChatGPT's table comparing GenAI versus DisAI.

Feature	Generative Models	Discriminative Models
Learns	$P(x, y)$	$P(y\|x)$
Generates new samples?	Yes	No
Typical tasks	Text, image generation; semi-supervised learning	Classification, regression
Complexity	High	Lower
Examples	GPT, GANs, VAEs, diffusion	Logistic regression, SVMs, random forests

Generative versus Discriminative AI Figure (ChatGPT)

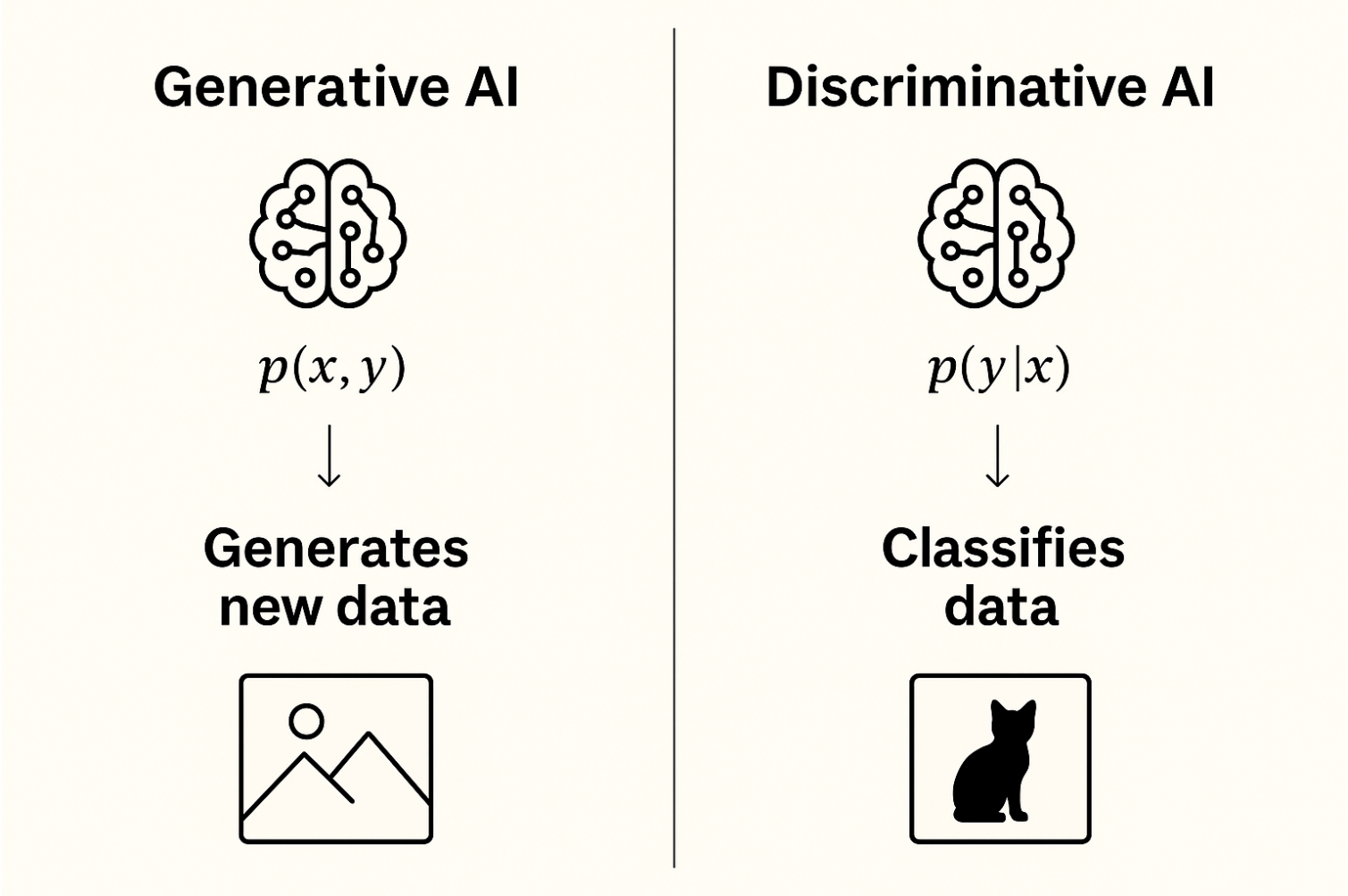

Figure 2: ChatGPT's visual diagram of GenAI versus DisAI.

Generative versus Discriminative AI (Claude)

Discriminative AI learns to distinguish between different categories or make predictions based on input data. It learns the boundary between classes. Think of it as answering "*which category does this belong to?*" or "*what's the relationship between these variables?*" Examples:

- Spam filters that classify emails as spam or not spam
- Image classifiers that identify whether a photo contains a cat or dog

Generative AI learns the underlying patterns and structure of the data itself, enabling it to create new examples. It models how the data is distributed and can answer "*what would a new example look like?*" Examples:

- Language models that write text
- Image generators that create pictures from descriptions

Technical difference:
Discriminative models learn $P(Y|X)$ - the probability of a label Y given input X
Generative models learn $P(X)$ or $P(X, Y)$ - the probability distribution of the data

Generative versus Discriminative AI Table (Claude)

Table 2: Claude's table comparing GenAI versus DisAI.

Aspect	Discriminative AI	Generative AI
Primary Function	Classifies or predicts	Creates new content
Key Question	"Which category is this?"	"What would a new example look like?"
What It Learns	Boundaries between categories	Underlying data patterns and distributions
Outputs	Labels, classifications, predictions	New text, images, audio, code, etc.
Technical Focus	$P(Y\|X)$ - probability of label given input	$P(X)$ or $P(X,Y)$ - probability distribution of data
Examples	Spam filters, image classifiers, fraud detection, recommendation systems	ChatGPT, DALL-E, Midjourney, code generators, music composers
Typical Use Cases	Decision-making, categorization, anomaly detection	Content creation, data augmentation, creative assistance
Training Goal	Maximize accuracy in distinguishing classes	Learn to generate realistic new samples

Note: I had to ask Claude to make a table summarizing the key differences, whereas ChatGPT and Gemini produced it automatically.

Generative versus Discriminative AI Figure (Claude)

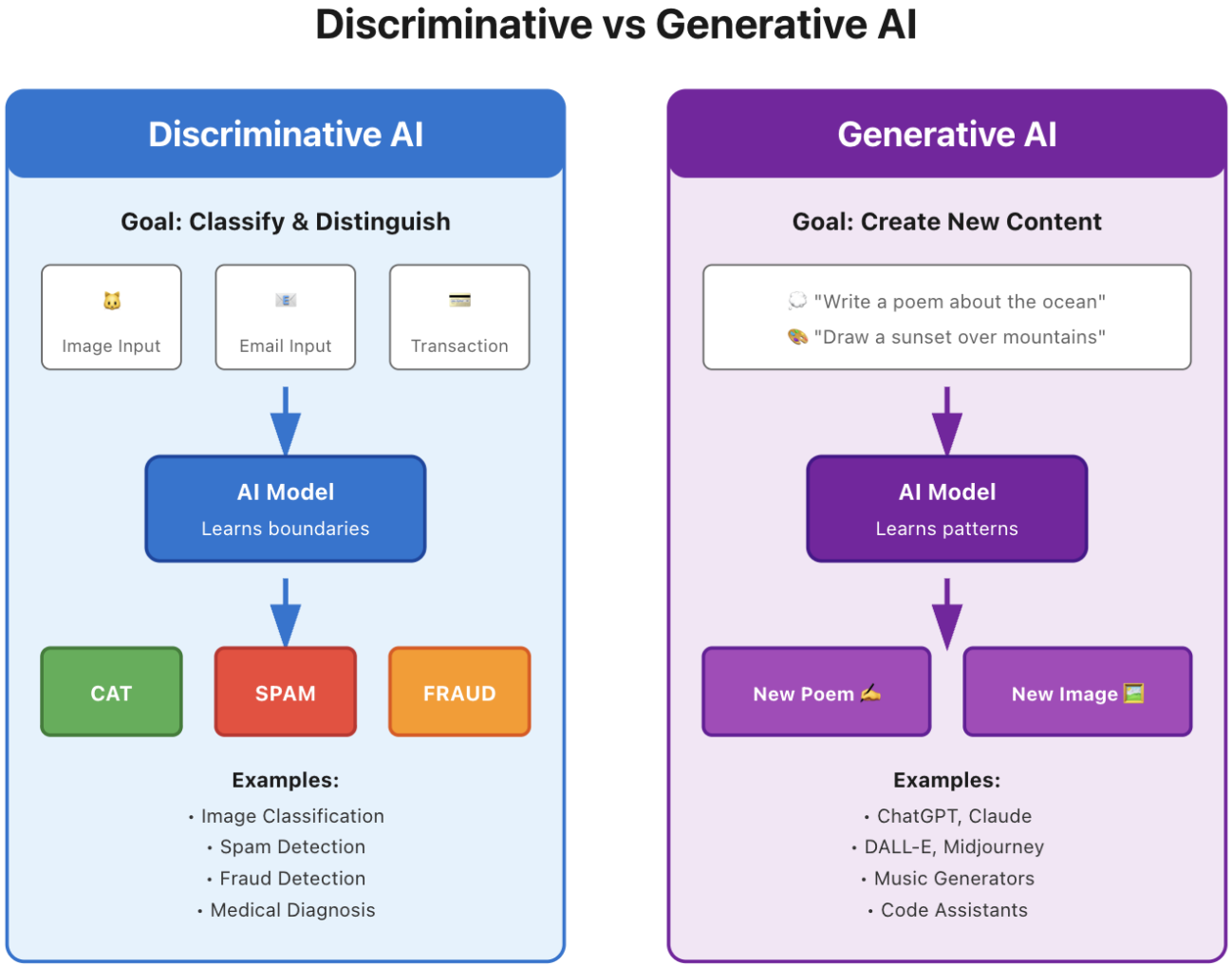

Figure 3: Claude's visual diagram of GenAI versus DisAI.

Generative versus Discriminative AI (Gemini)

Generative AI: The Creator

Generative models learn the complete probability distribution of the data. They ask: "*How was this data generated?*"

- Primary Goal: To generate new data instances (e.g., text, images, audio) that resemble the training data.
- Focus: Modeling the joint probability distribution of the input features (X) and the output classes (Y), which is $P(X, Y)$.
- Examples of Models: LLMs like GPT and Gemini, Generative Adversarial Networks (GANs) and Diffusion Models (for image/video generation), Naive Bayes (a classical generative classifier)

Discriminative AI: The Classifier

Discriminative models focus on modeling the conditional probability of a label given the input. They ask: "*What is the most likely class for this input?*"

- Primary Goal: To classify or predict the category of a given data input.
- Focus: Modeling the conditional probability distribution of the output class (Y) given the input features (X), which is $P(Y|X)$.
- Examples of Models: Logistic Regression, Support Vector Machines (SVMs), classical Deep Neural Networks (DNNs)

Generative versus Discriminative AI Table (Gemini)

Table 3: Gemini's table comparing GenAI versus DisAI.

Aspect	Generative AI (Creator)	Discriminative AI (Classifier)
Main Goal	Generate new, realistic content.	Classify or predict a label for existing input.
Mathematical Focus	Models the joint probability $P(X, Y)$.	Models the conditional probability $P(Y\|X)$
What it Learns	The entire underlying data distribution (how the data is formed).	The decision boundary (the line separating the classes).
Typical Tasks	Content creation, synthesis, simulation.	Classification, regression, prediction.
Example	Creating a brand new photo of a dog.	Determining if a given photo is of a dog or a cat.

Generative versus Discriminative AI Figure (Gemini)

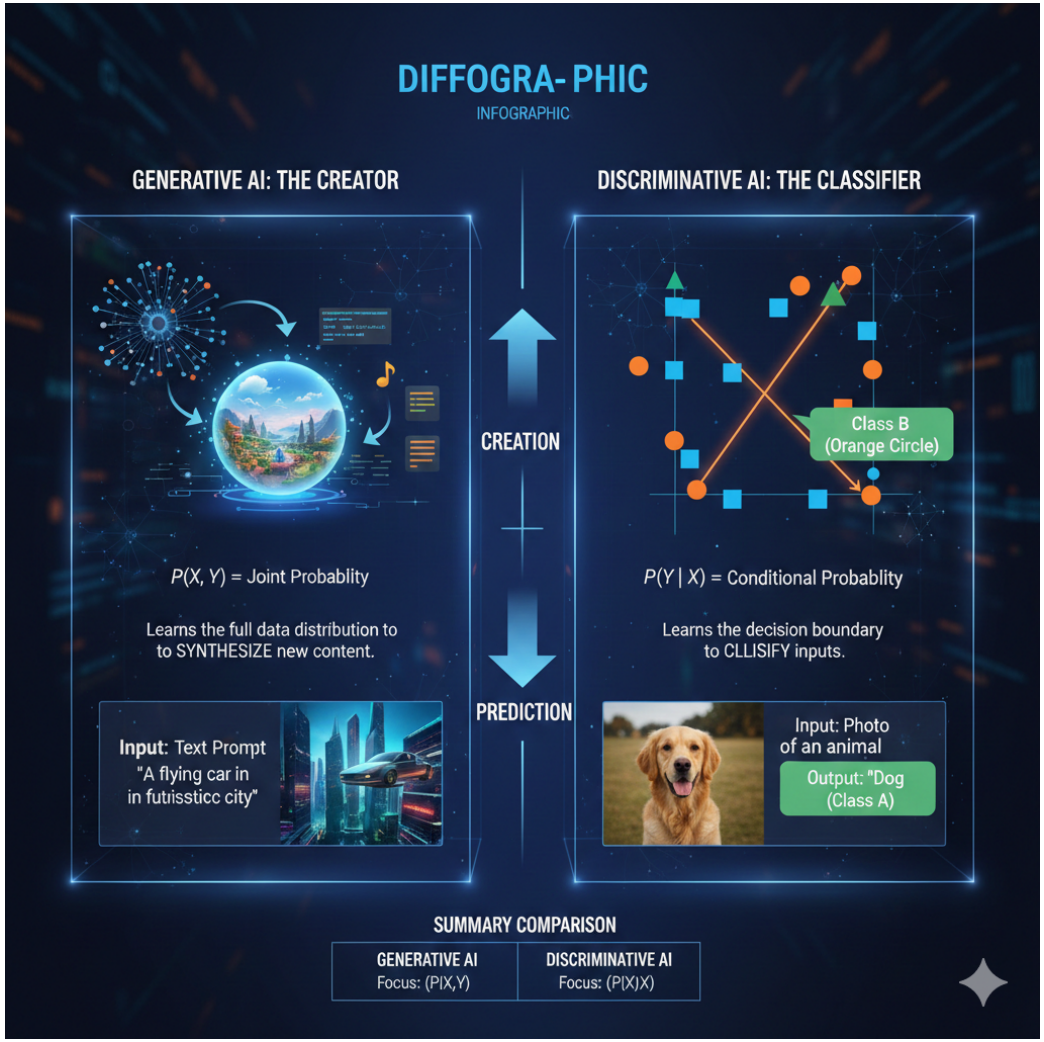

Figure 4: Gemini's visual diagram of GenAI versus DisAI.

Table of Contents

What are Large Language Models (LLMs)?

ChatGPT:
Large Language Models (LLMs) like GPT are neural networks trained to understand and generate human language. They work by learning statistical patterns from massive amounts of text and then using those patterns to predict what comes next in a sequence.

Claude:
Large Language Models (LLMs) are AI systems trained to understand and generate human language. LLMs learn patterns in language by analyzing massive amounts of textbooks, websites, articles, code, and more. They learn statistical relationships between words and concepts, enabling them to predict what text should come next in any given context.

Gemini:
Large Language Models (LLMs) are deep learning models that process and generate human language by operating as giant statistical prediction machines. Their core function is to predict the most likely next token (a word or sub-word) in a sequence, based on the input they are given and the vast amount of text they were trained on.

The Process LLMs Use to Generate Text (ChatGPT)

When you prompt an LLM:

1. Text is converted into tokens
2. Tokens are fed through the transformer layers
3. The model outputs a probability distribution for the next token
4. A token is sampled (based on temperature, top-k, nucleus sampling, etc.)
5. The new token is appended to the sequence
6. Steps 1-5 repeat until the model stops

Illustration of Large Language Model (ChatGPT)

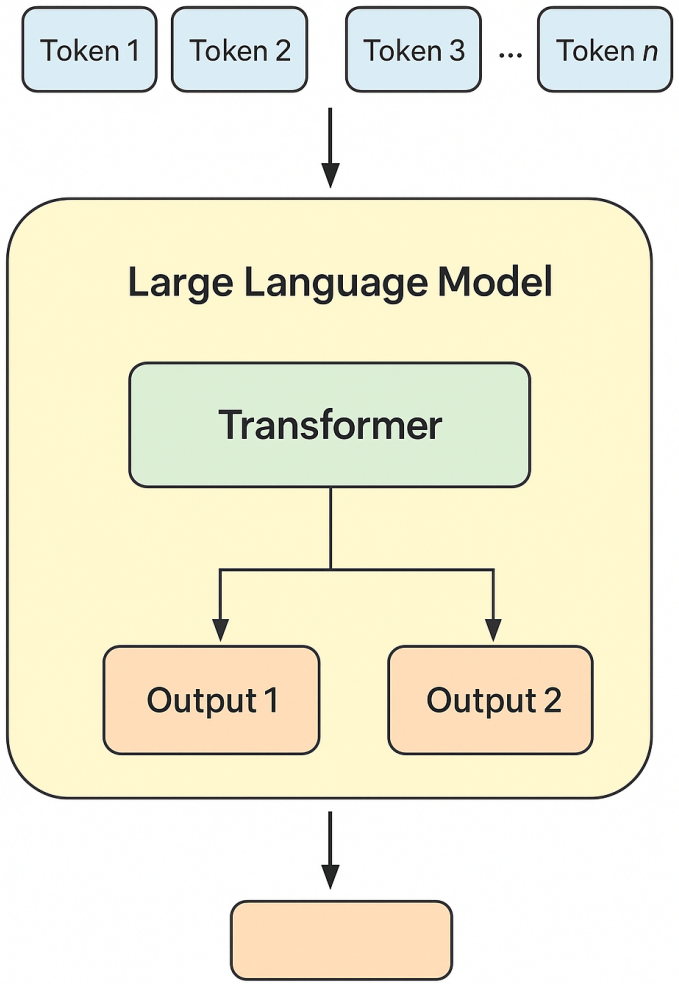

Figure 5: ChatGPT's illustration of large language models (LLMs).

The Process LLMs Use to Generate Text (Claude)

When you give an LLM a prompt, it:

1. Processes your input to understand the context
2. Predicts probability distributions for what word should come next
3. Selects a word (sometimes randomly from high-probability options)
4. Adds that word to the context and repeats the process
5. Continues until it generates a complete response

Think of it like an incredibly sophisticated autocomplete that understands context deeply enough to maintain coherence over long passages.

Illustration of Large Language Model (Claude)

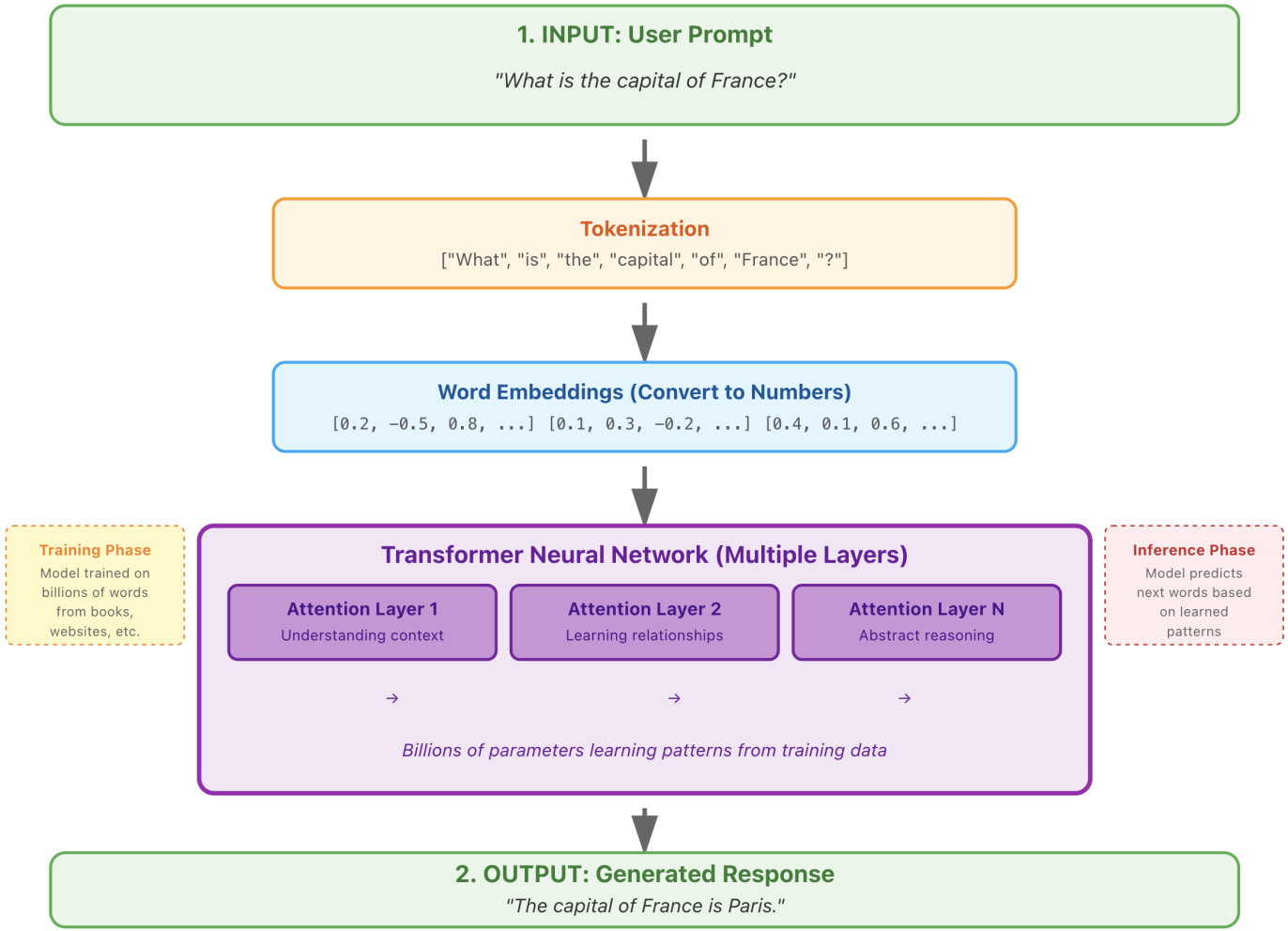

Figure 6: Claude's illustration of large language models (LLMs).

The Process LLMs Use to Generate Text (Gemini)

When you give an LLM a prompt, it goes through a cycle to generate the answer:

1. Contextual Understanding (Encoding)
   - Your prompt is Tokenized and Embedded
   - Transformer layers apply the attention mechanism to these numerical embeddings to create contextual numerical representation
2. Next-Token Prediction (Decoding)
   - Model uses this contextual representation to predict the probability of every possible token in its vocabulary appearing next
   - Model then selects one of the tokens based on these probabilities (randomness controlled by "temperature")
3. Iterative Generation (Autoregression)
   - Newly generated token is added to the original prompt
   - New, longer sequence is fed back into the model as the new input
   - Model repeats the process (Prediction $\rightarrow$ Selection $\rightarrow$ Addition) hundreds or thousands of times

Illustration of Large Language Model (Gemini)

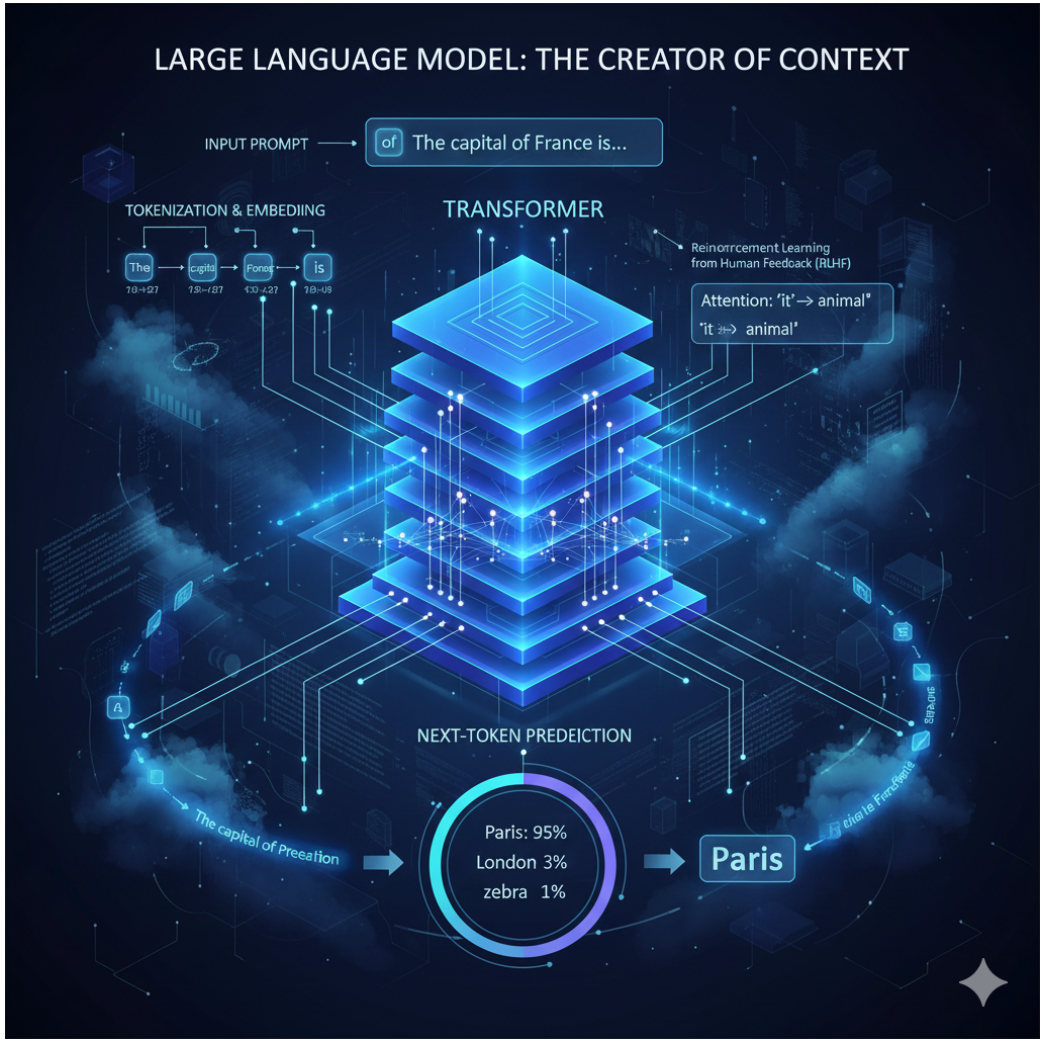

Figure 7: Gemini's illustration of large language models (LLMs).

Table of Contents

Key Components of Transformer Models

ChatGPT:
Almost all modern LLMs use the Transformer architecture, introduced in 2017.

Key components (a) Embeddings, (b) Self-attention Mechanism, (c) Layers of Transformers

Claude:
Modern LLMs use a "transformer" architecture, which has a few key features:

Attention Mechanism, Layers of Processing, Embeddings

Gemini:
The power of modern LLMs comes from a specific neural network architecture called the Transformer. This architecture allows the model to process large sequences of data (like a sentence or document) much more efficiently and effectively than previous methods

Tokenization and Embedding

ChatGPT:

Words/subwords are converted into dense numerical vectors.

Claude:

Words are converted into numerical representations that capture their meaning and relationships to other words.

Gemini:

Tokenization: The first step is breaking the input text (your prompt) into smaller, numerical units called tokens. A token might be a whole word ('the'), a part of a word ('ing'), a punctuation mark, or a number. *Embedding*: Each token is then converted into a numerical vector called an embedding. This vector is a list of numbers that represents the token's semantic meaning and context.

Self-Attention Mechanism Explained

ChatGPT:

This is the core innovation. Self-attention allows the model to dynamically determine which words in the input are relevant to each other and how strongly they relate regardless of distance in the sentence.

Claude:

The model learns which words in a sentence are most important for understanding context.

Gemini:

This is the heart of the Transformer. The self-attention mechanism allows the model to look at the entire input sequence at once and decide how much importance (or 'attention') to give to every other word when processing a single word. The Transformer uses this mechanism across multiple stacked layers to build a progressively more abstract and nuanced understanding of the input text.

Self-Attention Mechanism Examples

ChatGPT:

Example: In the sentence ‘The dog that chased the cats was hungry,’ attention helps the model connect ‘dog’ with ‘was hungry.’

Claude:

For example, in ‘The animal didn’t cross the street because it was too tired,’ the model learns that ‘it’ refers to ‘animal’ not ‘street.’

Gemini:

For example, in the sentence ‘The animal didn’t cross the street because it was too tired,’ the attention mechanism links the pronoun ‘it’ back to the subject, ‘animal,’ allowing the model to understand the correct context and meaning.

Self-Attention Mechanism Details (ChatGPT)

Step 1 — Convert each token embedding into Q, K, V
For each input token vector $x_i \in \mathbb{R}^d$:

$$Q_i = W_Q x_i, \qquad K_i = W_K x_i, \qquad V_i = W_V x_i$$

where

- W_Q, W_K, and W_V are learned matrices
- Q (Query): "What information do I need?"
- K (Key): "What information do I have to offer?"
- V (Value): "What is the information itself?"

Step 2 — Compute attention scores
The "relevance" of token j to token i is: $\text{score}(i,j) = \frac{Q_i K_j}{\sqrt{d_k}}$
The denominator prevents the dot product from blowing up.

Self-Attention Mechanism Details (ChatGPT con.)

Step 3 — Softmax to create probabilities

$$\alpha_{i,j} = \text{softmax}_j(\text{score}(i, j))$$

These are attention weights (how much token i pays attention to j).

Step 4 — Weighted sum of values
The output for token i is:

$$\text{Attention}(i) = \sum_j \alpha_{i,j} V_j$$

This produces a new vector that blends information from other tokens.

Self-Attention Mechanism Visualization (ChatGPT)

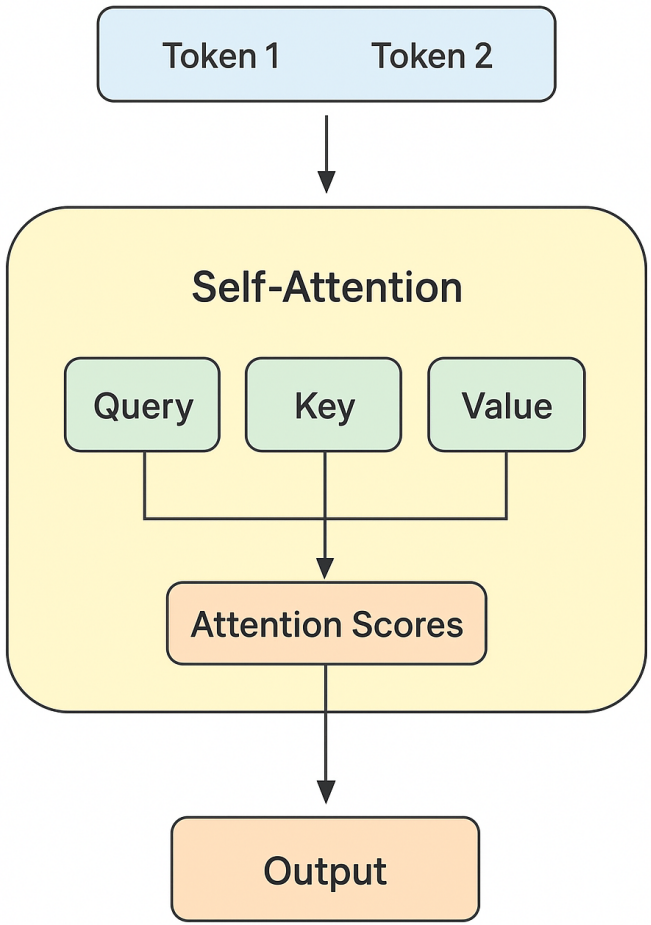

Figure 8: ChatGPT's illustration of the self-attention mechanism.

Self-Attention Mechanism Details (Claude)

The self-attention mechanism is the core innovation that makes transformers (and thus modern LLMs) so powerful.

Self-attention allows the model to weigh the importance of different words in a sequence when processing each word. For example, in "The animal didn't cross the street because it was too tired," self-attention helps the model figure out that "it" relates strongly to "animal" rather than "street."

Self-attention operates through three learned linear transformations that create Query (Q), Key (K), and Value (V) matrices:

Self-Attention Mechanism Details (Claude Con.)

Step 1: Create Q, K, V matrices
For each word's embedding vector $\mathbf{x}$, we compute:

- $\mathbf{Q} = \mathbf{x}\mathbf{W}^Q$ (Query: "what am I looking for?")
- $\mathbf{K} = \mathbf{x}\mathbf{W}^K$ (Key: "what do I contain?")
- $\mathbf{V} = \mathbf{x}\mathbf{W}^V$ (Value: "what information do I pass forward?")

Where $\mathbf{W}^Q$, $\mathbf{W}^K$, and $\mathbf{W}^V$ are learned weight matrices.

Step 2: Calculate attention scores
The attention score between words i and j is calculated as:

$$\text{score}(i, j) = Q_i \cdot K_j / \sqrt{d_k}$$

Where:

- $Q_i \cdot K_j$ is the dot product (measures similarity)
- d_k is the dimension of the key vectors (typically 64 or 128)
- Division by $\sqrt{d_k}$ prevents the dot products from becoming too large

Self-Attention Mechanism Details (Claude Con.)

Step 3: Apply softmax
Convert scores to probabilities that sum to 1:

$$\text{attention\_weights}(i, j) = \text{softmax}(\text{score}(i, j))$$

This gives us values between 0 and 1 that represent how much word i should "attend to" word j.

Step 4: Compute weighted sum
The output for word i is a weighted combination of all value vectors:

$$\text{output}_i = \sum_j (\text{attention\_weights}(i, j) \times V_j)$$

In Matrix Form: For efficiency, this is computed for all words simultaneously:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})V$$

This single equation captures the entire self-attention operation.

Self-Attention Mechanism Visualization (Claude)

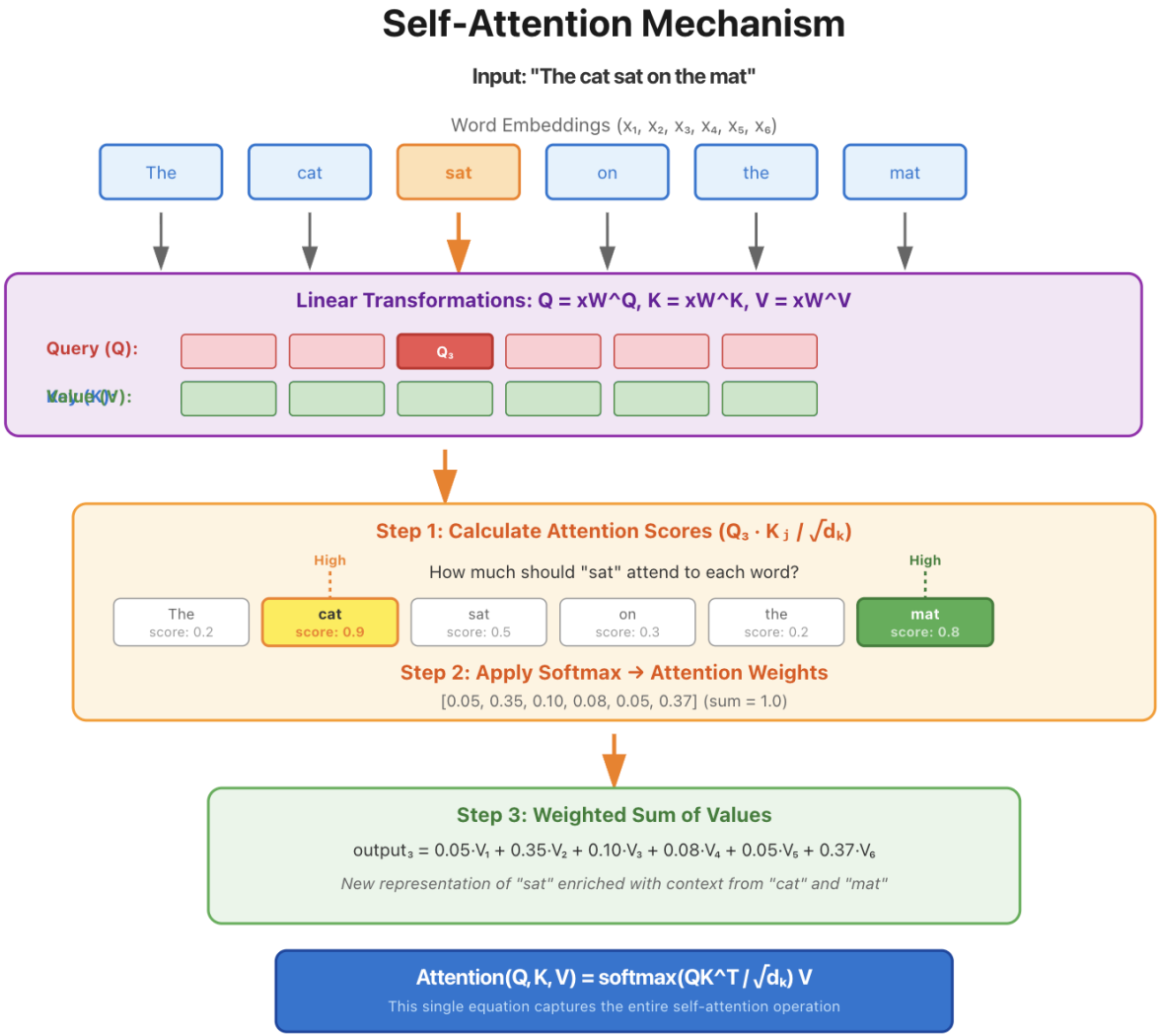

Figure 9: Claude's illustration of the self-attention mechanism.

Self-Attention Mechanism Details (Gemini)

The math behind the Self-Attention mechanism, specifically the Scaled Dot-Product Attention used in the Transformer architecture, is an elegant matrix computation that calculates the relevance of every element in an input sequence to every other element.

It is expressed in a single matrix equation:

$$\text{Attention}(Q, K, V) = \text{SoftMax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

This process can be broken down into five fundamental steps involving three key matrices: Query (Q), Key (K), and Value (V).

Self-Attention Mechanism Details (Gemini Con.)

1. Generating Q, K, and V Matrices

The input to the self-attention layer is a matrix, $\mathbf{X}$, which is the sequence of token embeddings (and positional encodings). This input matrix is linearly transformed using three learned weight matrices: $\mathbf{W}^Q$, $\mathbf{W}^K$, and $\mathbf{W}^V$.

$$Q = XW^Q, \qquad K = XW^K, \qquad V = XW^V$$

- **Q** (Query): Represents what the current token is looking for in other tokens.
- **K** (Key): Represents what the current token has to offer to other tokens for relevance comparison
- **V** (Value): Contains the actual content/information of the token to be aggregated in the final output.

The weight matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, and $\mathbf{W}^V$ are learnable parameters, allowing the model to project the same input information into three different semantic spaces for the attention calculation.

Self-Attention Mechanism Details (Gemini Con.)

2. Calculating the Alignment Score (Dot Product)
The next step calculates the raw attention score between every query and every key. This is a measure of their semantic alignment or similarity. This is done by multiplying the Query matrix (Q) by the transpose of the Key matrix (K^T):

$$\text{Score Matrix} = QK^T$$

The resulting score matrix, often called the Alignment Matrix, has a size of (sequence length $\times$ sequence length). Each element (i, j) in this matrix represents the raw relevance of the j-th token (Key) to the i-th token (Query).

3. Scaling
The raw attention scores are divided by the square root of the key vector's dimension, $\sqrt{d_k}$.

$$\text{Scaled Scores} = \frac{QK^T}{\sqrt{d_k}}$$

Purpose: This scaling factor prevents the dot products from becoming too large in magnitude when the dimension of the vectors (d_k) is high.

Self-Attention Mechanism Details (Gemini Con.)

4. Normalization (SoftMax)
The scaled scores are passed through the SoftMax function, applied row-wise.

$$\text{Attention Weights} = \text{SoftMax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

Purpose: The SoftMax function converts the raw scores into a set of positive probabilities (Attention Weights) that sum to 1 for each query vector.

5. Computing the Contextual Output
The final output of the self-attention layer is a weighted sum of the Value matrices, where the weights are the normalized attention scores.

$$\text{Attention Output} = \text{SoftMax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Result: The resulting Attention Output matrix is a new, context-aware representation of the input sequence.

Self-Attention Mechanism Visualization (Gemini)

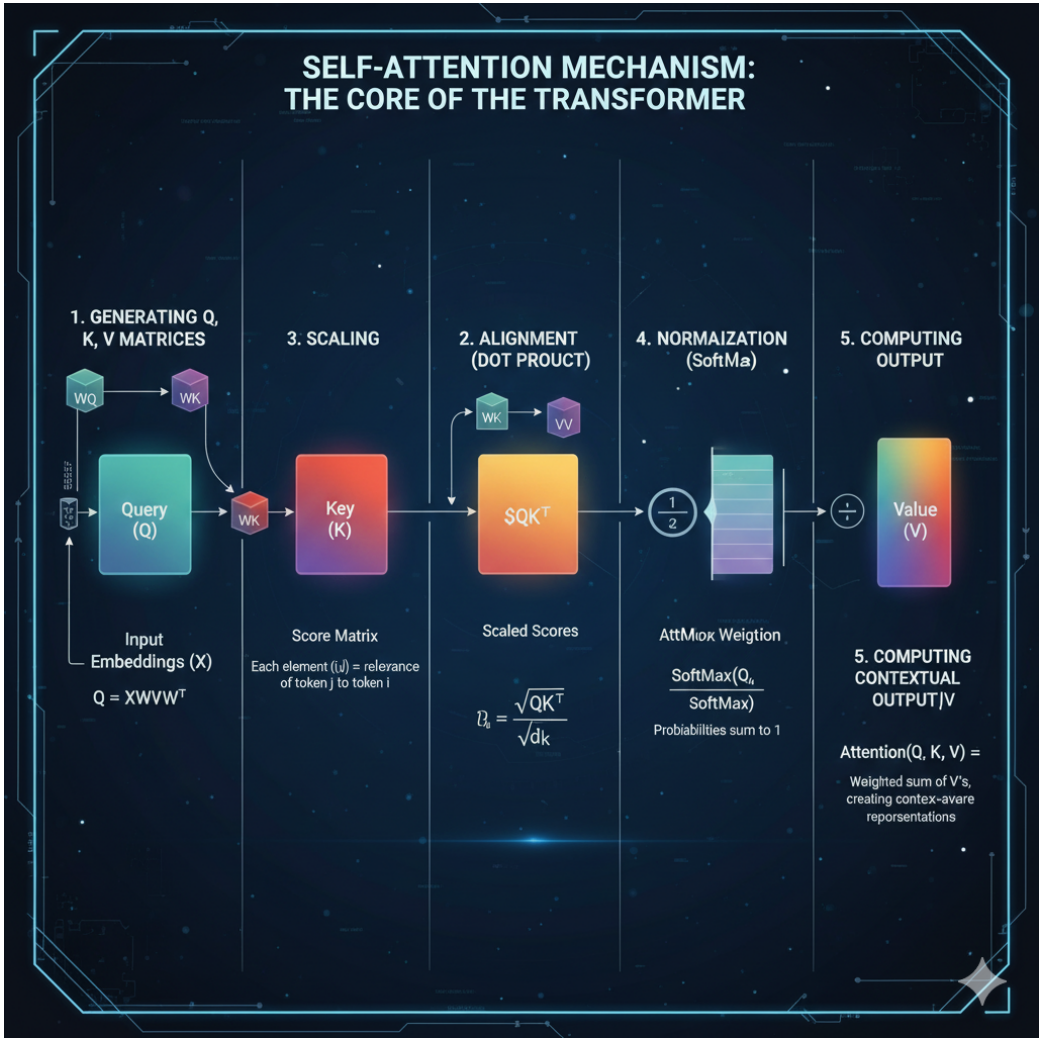

Figure 10: Gemini's illustration of the self-attention mechanism.

Table of Contents

Why LLMs Work So Well (ChatGPT)

Scale

- More parameters → richer representations
- More training data → broader knowledge
- More compute → better optimization

Self-Supervision

- The model learns from text with out human labeling.
- Next-token prediction is a naturally abundant training signal.

Generalization

- Learn latent structures of language that transfer to many tasks: translation, summarization, reasoning, question answering, coding
- They can perform these tasks zero-shot (without specific training) because they learned general language patterns.

Fine-Tuning and Alignment (ChatGPT)

After pretraining, LLMs can be:

- *Fine-tuned*: On specific tasks (e.g., coding, math, medical Q&A)
- *Instruction-tuned*: Trained to follow user instructions via supervised datasets.
- *RLHF (Reinforcement Learning from Human Feedback)*: Humans rank model outputs; model learns which behaviors are preferred.

These steps make the model helpful, safe, and aligned with human expectations.

Some Unique Comments the Agents Made about LLMs

ChatGPT on What the Model Actually Learns:

LLMs do **not** memorize specific documents (though they may memorize small pieces). They learn grammar, world knowledge, reasoning patterns, statistical associations, writing styles, relationships between concepts. The model stores this knowledge in billions of parameters (weights).

Claude on Important Limitations:

LLMs don't truly 'understand' in a human sense - they're pattern-matching systems. They can produce text that sounds authoritative but may be incorrect (called 'hallucinations'). They also only know what was in their training data and can reflect biases present in that data.

Gemini on Why LLMs are Large:

(1) Vast Training Data: They are initially trained on massive amounts of text data—trillions of words scraped from the internet, books, and articles—allowing them to learn the statistical properties of human language, grammar, and a wide range of facts.
(2) Billions of Parameters: The model's capacity is determined by its number of parameters. . . Modern LLMs have billions or even trillions of these parameters, which is what gives them their immense power to understand complex relationships.

LLMs in Scientific Research Pipeline (ChatGPT)

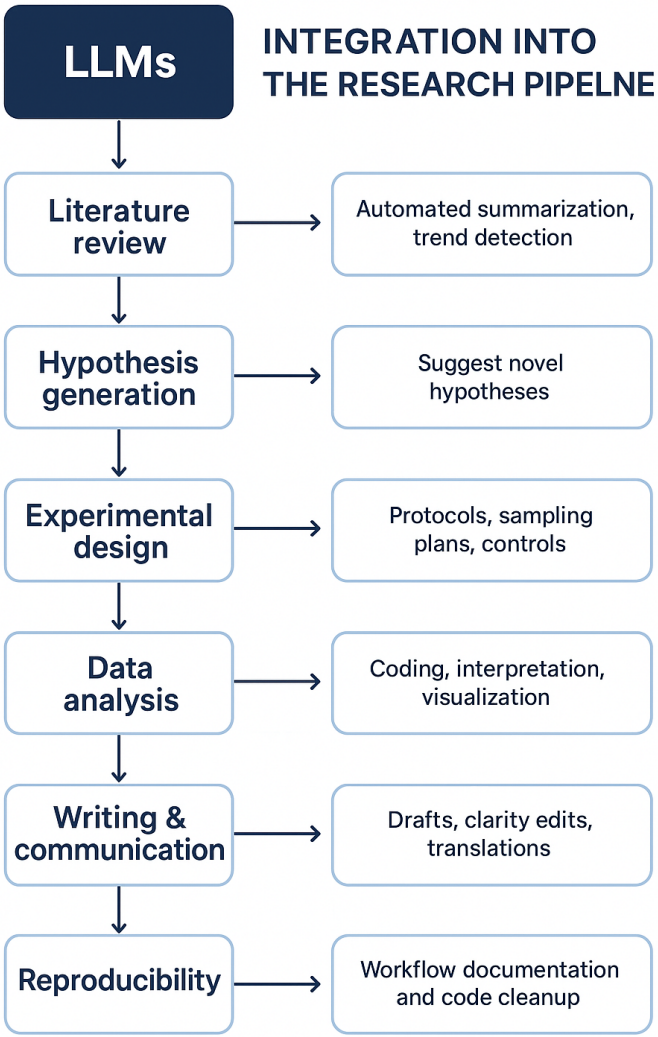

Figure 11: ChatGPT's illustration of how LLMs can be used in scientific research.

Ten ways to use GenAI and LLMs for science:

1. Accelerating Literature Review & Knowledge Synthesis
2. Improving Experimental Design
3. Automating Data Analysis and Interpretation
4. Generating Synthetic Data
5. Enhancing Scientific Writing and Communication
6. Assisting with Coding, Simulation, and Modeling
7. Supporting Reproducibility and Transparency
8. Enhancing Simulation and Discovery in the Natural Sciences
9. Brainstorming and Creativity Enhancement
10. Democratizing Access to Expertise

References

Anthropic (2025). Claude (Model Sonnet 4.5). [Large Language Model] Accessed 2025-12-02. `https://claude.ai/`
Claude Chat History

Google (2025). Gemini (Model 2.5 Flash). [Large Language Model] Accessed 2025-12-02. `https://gemini.google.com/`
Gemini Chat History

OpenAI (2025). ChatGPT (Model GPT-5.1). [Large Language Model] Accessed 2025-12-02. `https://chat.openai.com`
ChatGPT Chat History